

# Embedded Linux & System Software Interview Handbook

## Chapter 5 - Linux Device Drivers (Part 5)

**Topics:** Interrupt Handling, ISR, Top Half, Bottom Half, Tasklets, Workqueues and Threaded IRQs

### 5.8 Introduction to Interrupts

An interrupt is a hardware or software event that temporarily stops the CPU's current execution so that urgent work can be processed immediately. Interrupts eliminate the need for continuous polling and improve overall system responsiveness.

#### *Polling vs Interrupts*

| Polling                             | Interrupt                              |
|-------------------------------------|----------------------------------------|
| CPU repeatedly checks device status | Device notifies CPU only when required |
| Higher CPU utilization              | Efficient CPU usage                    |
| Higher latency                      | Lower latency                          |
| Suitable for simple systems         | Preferred for modern embedded systems  |

### Interrupt Flow

Hardware Event → Interrupt Controller (GIC/APIC) → CPU → Interrupt Service Routine (ISR) → Return to Interrupted Task

### 5.9 Registering an Interrupt

The Linux kernel provides `request_irq()` to register an interrupt handler. The handler is invoked whenever the corresponding interrupt occurs.

#### *Typical API*

```
request_irq(irq_number, handler, flags, "driver_name", dev);
```

### 5.10 Interrupt Service Routine (ISR)

An ISR executes in interrupt context and should finish as quickly as possible. It acknowledges the interrupt, captures essential information and schedules deferred work if additional processing is required.

#### *Best Practices*

- Keep ISR short.
- Avoid sleeping.
- Avoid lengthy calculations.
- Protect shared data.
- Schedule deferred work when necessary.

### 5.11 Top Half vs Bottom Half

**Top Half:** Immediate interrupt handling (ISR).

**Bottom Half:** Deferred processing performed after the ISR completes.

### ***Bottom Half Mechanisms***

| Mechanism    | Typical Usage               |
|--------------|-----------------------------|
| SoftIRQ      | High-performance networking |
| Tasklet      | Short deferred work         |
| Workqueue    | Operations that may sleep   |
| Threaded IRQ | Longer interrupt processing |

## **5.12 Threaded Interrupts**

Threaded IRQs execute part of the interrupt processing in a kernel thread. They are useful when interrupt handling requires sleeping or more complex processing.

### **Real Example**

A GPIO button interrupt may simply record the interrupt in the ISR and schedule a workqueue to debounce the button, update device state and notify user space.

### **Debugging Interrupt Problems**

- Check `/proc/interrupts`.
- Verify interrupt count increases.
- Inspect `dmesg` for IRQ registration errors.
- Validate Device Tree interrupt properties.
- Confirm hardware interrupt polarity and trigger type.

### **Common Interview Questions**

1. What is an interrupt?
2. Polling vs Interrupts?
3. Why should an ISR be short?
4. Difference between Top Half and Bottom Half?
5. When would you use a Workqueue instead of a Tasklet?
6. What is `request_irq()`?
7. How do you debug a missing interrupt?

### **Key Takeaways**

Efficient interrupt handling is fundamental to Linux driver development. Senior engineers minimize ISR execution time, defer non-critical work appropriately and use kernel debugging tools to diagnose interrupt-related issues.